

# Test Implementation Plan v2 — Comprehensive Revisions

---

**Date:** 2026-05-27

**Document:** Response to 22-item plan review (items A1 through K2)

**Baseline:** TEST\_IMPLEMENTATION\_PLAN.md (commit `d69f8ab` )

**Workspace:** `/home/ubuntu/test_workspace/`

---

## How to Read This Document

---

Each review item gets three things:

1. **Position** — AGREE, PARTIALLY AGREE, or DISAGREE (with evidence)
2. **What the audit actually found** — real data from the codebase, not assumptions
3. **Concrete plan change** — what the v2 plan will do differently

Items are grouped by the original review sections (A through K).

---

## SECTION A: Effort Estimate Reality Check

---

### A1: “6 minutes per test” Is Unrealistic

**Position:** AGREE. The v1 estimate was naïve.

**Evidence from this workspace:**

The existing 153 tests were written over multiple sessions totaling ~40 hours of agent time (including debugging, fixture creation, mock development, and environment setup). That’s **~16 minutes per test** for an established pattern. New tests in untested areas will be slower because:

- No established fixture patterns for backup, manifests, WebSocket
- Mock infrastructure incomplete (Redis pub/sub, subprocess)
- Bug discoveries (see Section H) will interrupt flow
- Service tests need classification before patterns emerge (see D1)

**Revised Effort Model:**

| Test Category                   | Time per Test | Justification                     |
|---------------------------------|---------------|-----------------------------------|
| API CRUD (happy path)           | 10 min        | Pattern exists, copy+adapt        |
| API error paths (404, 403, 422) | 8 min         | Simple variations                 |
| Rate limit tests                | 20 min        | New mock patterns needed          |
| WebSocket tests                 | 30 min        | Unproven approach, async timing   |
| Service unit (pure DB)          | 15 min        | Direct DB, no HTTP layer          |
| Service unit (external deps)    | 25 min        | Mock setup per external dep       |
| Integration tests               | 40 min        | Container setup, teardown         |
| First test in new file          | 45 min        | Fixture setup, imports, debugging |

#### Revised Phase Estimates:

| Phase                  | Tests      | Avg Min/ Test | Est. Hours | Contingency | Total      |
|------------------------|------------|---------------|------------|-------------|------------|
| Phase 0: Migrations    | N/A        | N/A           | 8h         | +4h         | <b>12h</b> |
| Phase 1: Rate Limiting | 15         | 20            | 5h         | +3h         | <b>8h</b>  |
| Phase 2: WebSocket     | 12         | 30            | 6h         | +4h         | <b>10h</b> |
| Phase 3: API Endpoints | 33         | 12            | 7h         | +3h         | <b>10h</b> |
| Phase 4: Services      | 89         | 18            | 27h        | +9h         | <b>36h</b> |
| Phase 6: Integration   | 20         | 40            | 13h        | +5h         | <b>18h</b> |
| <b>TOTAL</b>           | <b>169</b> | —             | <b>66h</b> | <b>+28h</b> | <b>94h</b> |

#### Confidence Intervals:

| Scenario                    | Hours | Probability |
|-----------------------------|-------|-------------|
| Best case (no surprises)    | 66h   | 20%         |
| Expected (normal issues)    | 94h   | 50%         |
| Worst case (major blockers) | 130h  | 90%         |

**Plan Change:** Replace all point estimates with ranges. Add explicit contingency per phase. Use 94h as the planning number.

---

## A2: Confidence Intervals

**Position:** AGREE. Ranges beat points.

### What drives high-end estimates:

1. WebSocket POC fails → need alternative approach (+8h)
2. Schema mismatches discovered during testing → investigation + fixes (+12h)  
**UPDATE:** Already found 8 bugs (see Section H). This risk is REALIZED.
3. Service classification reveals more context-dependent services than expected (+6h)
4. Integration test containers fail in sandbox (+8h)

**Plan Change:** Every phase gets a “halt-and-report” gate. If contingency time is consumed, stop and report status before continuing.

---

## SECTION B: Schema Changes Without Migrations (CRITICAL)

---

### B1: Phase 0 Is Mandatory

**Position:** STRONGLY AGREE. And it’s worse than v1 stated.

#### Audit Results (verified against actual files):

The v1 plan identified 4 missing columns. The actual audit found **5 columns missing from migrations** plus **6 additional production bugs** the v1 plan completely missed.

**Migration Gaps (columns in models, not in any migration):**

| Table                 | Column      | Model File          | Impact                                     |
|-----------------------|-------------|---------------------|--------------------------------------------|
| users                 | is_active   | user.py             | alembic upgrade head will miss this column |
| projects              | created_by  | project.py          | FK to users — deployment blocker           |
| as-set_quality_scores | job_id      | quality_score.py    | FK to render_jobs — deployment blocker     |
| retention_policies    | description | retention_policy.py | Text column — deployment will miss it      |
| prompt_tags           | description | prompt_tag.py       | Text column — deployment will miss it      |

**Schema Mismatches (API code vs. ORM model — NEWLY DISCOVERED):**

| Bug ID  | File               | Issue                                                                                                                                                                                         | Severity      |
|---------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| BUG-003 | manifests.py       | API raw SQL uses <code>timeline_json</code> , <code>scene_count</code> , <code>created_at</code> — model has <code>timeline</code> , no <code>scene_count</code> , no <code>created_at</code> | <b>HIGH</b>   |
| BUG-004 | manifests.py       | API uses <code>sha256_hash</code> on assets table — model column is <code>content_hash</code>                                                                                                 | <b>HIGH</b>   |
| BUG-005 | backup.py          | API uses <code>storage_path</code> — model column is <code>backup_path</code>                                                                                                                 | <b>HIGH</b>   |
| BUG-006 | backup.py          | API uses <code>error_message</code> — model has no such column                                                                                                                                | <b>HIGH</b>   |
| BUG-007 | quality_service.py | Service writes <code>score.review_notes</code> — model has no such attribute                                                                                                                  | <b>MEDIUM</b> |

**Critical finding:** `manifests.py` and `backup.py` — the two largest untested API files — have **fundamental column name mismatches** between their raw SQL and the ORM models. These endpoints cannot work in production as written. This means Phase 3 tests for these endpoints will fail immediately until the mismatches are resolved.

**All Tables vs. Migrations (complete):**

Every table in every model file has a corresponding migration:

|                           |                              |
|---------------------------|------------------------------|
| ✓ asset_quality_scores    | ✗ 0008_quality_scores        |
| ✓ assets                  | ✗ 0001_initial_core          |
| ✓ audit_log               | ✗ 0001_initial_core          |
| ✓ backup_records          | ✗ 0013_backup_records        |
| ✓ composition_manifests   | ✗ 0007_composition_manifests |
| ✓ dead_letter_messages    | ✗ 0006_dead_letter_queue     |
| ✓ fallback_policies       | ✗ 0014_fallback_policies     |
| ✓ gpu_metrics_history     | ✗ 0010_gpu_metrics           |
| ✓ gpu_nodes               | ✗ 0003_gpu_registry          |
| ✓ gpu_reservations        | ✗ 0003_gpu_registry          |
| ✓ language_variants       | ✗ 0001_initial_core          |
| ✓ pipeline_checkpoints    | ✗ 0002_pipeline_checkpoints  |
| ✓ projects                | ✗ 0001_initial_core          |
| ✓ prompt_tag_associations | ✗ 0001_initial_core          |
| ✓ prompt_tags             | ✗ 0001_initial_core          |
| ✓ prompts                 | ✗ 0001_initial_core          |
| ✓ render_jobs             | ✗ 0001_initial_core          |
| ✓ render_segments         | ✗ 0009_render_segments       |
| ✓ retention_policies      | ✗ 0011_retention_policies    |
| ✓ rollback_points         | ✗ 0001_initial_core          |
| ✓ storage_quotas          | ✗ 0012_storage_quotas        |
| ✓ storyboard_scenes       | ✗ 0001_initial_core          |
| ✓ task_retries            | ✗ 0004_retry_tracking        |
| ✓ transcripts             | ✗ 0001_initial_core          |
| ✓ users                   | ✗ 0001_initial_core          |
| ✓ worker_heartbeats       | ✗ 0005_worker_heartbeats     |

### Phase 0 Deliverables:

#### Phase 0a: Schema Audit (2h)

- ✗ SCHEMA\_MIGRATION\_GAP\_REPORT.md (DONE ✗ findings above)

#### Phase 0b: New Alembic Migrations (4h)

- ✗ 0015\_add\_users\_is\_active.py
- ✗ 0016\_add\_projects\_created\_by.py
- ✗ 0017\_add\_quality\_scores\_job\_id.py
- ✗ 0018\_add\_retention\_description.py
- ✗ 0019\_add\_prompt\_tags\_description.py

#### Phase 0c: Bug Fix Decisions (2h)

- ✗ For BUG-003 through BUG-007: decide fix-in-model vs fix-in-API
- ✗ Cannot write tests for manifests/backup until resolved

#### Phase 0d: Verification (2h)

- ✗ alembic upgrade head clean
- ✗ alembic downgrade base; alembic upgrade head
- ✗ 153 existing tests still pass

**Plan Change:** Phase 0 is now the first phase. Phase 3 (backup, manifests) is BLOCKED until Phase 0c resolves schema mismatches. Phase 0 estimate: 8-12h.

## B2: Exhaustive Schema Audit

**Position:** AGREE. Completed above.

**Plan Change:** Audit is done. Findings embedded in B1. No additional work needed — the audit was performed as part of this revision.

## SECTION C: WebSocket Testing Approach

### C1: Need Concrete Approach Before Phase 2

**Position:** AGREE. The v1 plan hand-waved this.

**Available Options (verified in this sandbox):**

| Approach                                                         | Available?          | Pros                      | Cons                                           |
|------------------------------------------------------------------|---------------------|---------------------------|------------------------------------------------|
| <code>starlette.testclient.TestClient.websocket_connect()</code> | ✅ Yes               | Sync, simple, built-in    | Sync only — may conflict with async test suite |
| <code>httpx + httpx-ws</code>                                    | ❌ Not installed     | Async, matches test style | Extra dependency                               |
| <code>websockets</code> library direct                           | ✅ Installed (v16.0) | Production-grade          | Needs running server                           |

**Recommended:** Starlette `TestClient.websocket_connect()`

Rationale:

- Already available (Starlette 0.41.3 installed)
- No new dependencies
- Well-documented pattern for FastAPI WebSocket testing
- Handles both connect/disconnect lifecycle

**Test Pattern:**

```
from starlette.testclient import TestClient
from main import app

def test_ws_node_logs_unknown_node():
    """Unknown node_id → error JSON + close 1008."""
    client = TestClient(app)
    with client.websocket_connect("/ws/nodes/unknown-node/logs") as ws:
        data = ws.receive_json()
        assert "error" in data
        assert "Unknown node" in data["error"]
    # Connection closed with code 1008
```

**Challenge: Subprocess Mocking**

Both WebSocket endpoints spawn subprocesses or connect to Redis. Tests must mock:

1. `asyncio.create_subprocess_shell` → Return mock process with controlled stdout
2. `redis.asyncio.from_url` → Return mock with pub/sub support

**POC Gate:**

Before starting Phase 2, run a 30-minute POC:

1. Connect to a WS endpoint with TestClient
2. Verify `websocket.accept()` works
3. Verify `send_json/receive_json` works
4. If fails → try `httpx-ws` (install + test, 30 min)
5. If both fail → halt and report

#### Plan Change:

- Phase 2 starts with mandatory 30-min POC
- Fallback sequence documented
- Halt gate if all approaches fail

## C2: Redis Pub/Sub Mocking

**Position:** AGREE. Current mock is minimal.

#### Current Redis Mock Capabilities (from `confest.py`):

|   |                |                                                       |
|---|----------------|-------------------------------------------------------|
| ✓ | get/set/delete | – basic key-value                                     |
| ✓ | exists         | – key existence check                                 |
| ✓ | incr           | – atomic increment                                    |
| ✓ | expire         | – NO-OP (always returns True, never expires)          |
| ✓ | ping/close     | – stubs                                               |
| ✗ | pubsub         | – NOT IMPLEMENTED                                     |
| ✗ | subscribe      | – NOT IMPLEMENTED                                     |
| ✗ | get_message    | – NOT IMPLEMENTED                                     |
| ✗ | from_url       | – NOT IMPLEMENTED (ws_logs.py creates its own client) |

#### Required Enhancement for Phase 2:

```
class MockRedisPubSub:
    """Minimal pub/sub mock for WebSocket job status tests."""

    def __init__(self):
        self._queues: dict[str, asyncio.Queue] = {}

    async def subscribe(self, channel: str):
        self._queues[channel] = asyncio.Queue()

    async def get_message(self, ignore_subscribe_messages=True, timeout=1.0):
        for ch, q in self._queues.items():
            try:
                data = await asyncio.wait_for(q.get(), timeout=timeout)
                return {"type": "message", "channel": ch, "data": data}
            except asyncio.TimeoutError:
                return None

    async def unsubscribe(self, channel: str):
        self._queues.pop(channel, None)

    # Test helper: inject a message
    async def inject(self, channel: str, data: str):
        if channel in self._queues:
            await self._queues[channel].put(data)
```

**Problem:** `ws_logs.py:114` creates its own Redis client via `aioredis.from_url(settings.REDIS_URL)` — it does NOT use the shared `redis_client` that the mock patches. This means the existing mock won't intercept it.

**Fix Options:**

1. Refactor `ws_logs.py` to use shared `redis_client` (production code change)
2. Monkeypatch `redis.asyncio.from_url` at module level in tests
3. Accept: test only `stream_node_logs`, skip `stream_job_status`

**Recommendation:** Option 2 (monkeypatch `redis.asyncio.from_url`).

**Plan Change:**

- Add pub/sub mock to Phase 2 setup tasks (2h)
- Document the `from_url` monkeypatch requirement
- Add this as a risk: "If monkeypatch doesn't intercept, fall back to Option 3"

## SECTION D: Direct Service Testing Pattern

### D1: Service Classification Audit

**Position:** AGREE. Classification is mandatory before Phase 4.

**Audit Results (verified against actual source):**

| Service                | Classification       | External Deps             | Direct Unit Test? |
|------------------------|----------------------|---------------------------|-------------------|
| check-point_service.py | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| dlq_service.py         | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| gpu_service.py         | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| job_service.py         | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| language_service.py    | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| prompt_service.py      | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| quality_service.py     | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| retention_service.py   | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| story-board_service.py | <b>PURE_DB</b>       | DB only                   | ✓ Yes             |
| asset_service.py       | <b>DB+EXTERNAL</b>   | DB, SeaweedFS             | ⚠ With mock       |
| transcript_service.py  | <b>DB+EXTERNAL</b>   | DB, SeaweedFS, filesystem | ⚠ With mock       |
| project_service.py     | <b>DB+EXTERNAL</b>   | DB, current_user context  | ⚠ With mock       |
| rollback_service.py    | <b>EXTERNAL_ONLY</b> | Subprocess, filesystem    | ⚠ Heavy mocking   |
| auth_service.py        | <b>EXTERNAL_ONLY</b> | Redis                     | ⚠ With mock       |
| user_service.py        | <b>PURE_LOGIC</b>    | None (static methods)     | ✓ Yes             |

**Key Finding:** 10 of 15 services are **PURE\_DB** — they take an `AsyncSession` constructor argument and nothing else. These are straightforward to test directly:

```
service = CheckpointService(db_session)
result = await service.list_checkpoints(job_id)
```

Only 3 need external mocking (SeaweedFS, subprocess), and 1 needs Redis.

`project_service.py` uses `current_user` but only for ownership checks — it can be tested by creating the user in the DB fixture.

**Plan Change:**

- Phase 4 starts with PURE\_DB services (10 services, fastest wins)
- DB+EXTERNAL services second (3 services, need mock setup)
- EXTERNAL\_ONLY last (rollback\_service needs heavy subprocess mocking)
- Classification table included in v2 plan

**D2: Rationale for Service-Level vs. API-Level Tests**

**Position:** AGREE. Each service test file needs justification.

**When service tests add value over API tests:**

| Reason                                    | Example                                                                 | API test sufficient? |
|-------------------------------------------|-------------------------------------------------------------------------|----------------------|
| Internal method not reachable via API     | <code>gpu_service._to_response()</code>                                 | No                   |
| Complex business logic with many branches | <code>retention_service.get_report()</code><br>tier calculation         | Partially            |
| Error paths hard to trigger via API       | <code>checkpoint_service.resume_from_checkpoint()</code> stage ordering | No                   |
| Performance-sensitive aggregation         | <code>gpu_service.get_fleet_utilization()</code>                        | Partially            |

**Services where API tests already cover adequately:**

Looking at existing test counts vs. service methods:

| Service            | Methods                       | API Tests                      | Gap?                                 |
|--------------------|-------------------------------|--------------------------------|--------------------------------------|
| checkpoint_service | 4                             | 15<br>(test_checkpoint_api.py) | ✗ Well-covered                       |
| dlq_service        | 6                             | 15 (test_dlq_api.py)           | ✗ Well-covered                       |
| gpu_service        | 7 + <code>_to_response</code> | 17 (test_gpu_api.py)           | ⚠ <code>_to_response</code> untested |
| quality_service    | 4                             | 11<br>(test_quality_api.py)    | ⚠ Avg score calc untested            |
| retention_service  | 5                             | 14<br>(test_retention_api.py)  | ⚠ Report logic untested              |
| project_service    | 8                             | 16 (test_projects.py)          | ⚠ Trigger pipeline untested          |

**Revised Approach:** Instead of creating a service test file for every service, create service tests ONLY where:

1. API tests cannot reach internal methods
2. Branch coverage of the service file is below 60% after API tests
3. Complex business logic needs isolated testing

**Estimated reduction:** From 13 service test files → 8 service test files. Services with good API coverage (checkpoint, dlq) get skipped unless coverage analysis shows specific untested branches.

#### Plan Change:

- After Phase 3 API tests complete, re-run coverage
- Only create Phase 4 service tests for files still under 60%
- Document which services are adequately covered by API tests

## SECTION E: Coverage Quality, Not Just Quantity

### E1: Branch Coverage Targets

**Position:** STRONGLY AGREE. Line coverage hides untested conditionals.

#### What Changes:

Add `--cov-branch` to all pytest coverage runs:

```

pytest tests/ \
  --cov=app \
  --cov-branch \
  --cov-report=term-missing \
  --cov-report=json \
  --cov-report=html

```

#### Targets:

| Metric                              | Threshold   | Gate Type                       |
|-------------------------------------|-------------|---------------------------------|
| Line coverage (overall)             | $\geq 80\%$ | Hard gate (\$15.4)              |
| Branch coverage (overall)           | $\geq 65\%$ | Soft gate (report, don't block) |
| Branch coverage (critical services) | $\geq 70\%$ | Hard gate                       |

**Critical services for branch coverage:** `rate_limit.py`, `gpu_service.py`, `rollback_service.py`, `quality_service.py`, `checkpoint_service.py`

**Why 65% branch (not 70%):** Branch coverage is typically 10-15 points lower than line coverage for the same test suite. Setting it too high makes it unachievable without heroic mocking of every error path.

#### Plan Change:

- All coverage commands include `--cov-branch`
- Branch coverage reported alongside line coverage
- Phase completion gates include both metrics
- v2 plan specifies branch targets explicitly

## E2: Critical Path Identification

**Position:** AGREE. Coverage is a proxy; critical paths are what matter.

#### 10 Critical Operational Paths (with test mapping):

| #  | Path                                             | Current Coverage                      | Phase     |
|----|--------------------------------------------------|---------------------------------------|-----------|
| 1  | POST /auth/login → token → authenticated request | ✅ Tested (12 auth tests)              | Existing  |
| 2  | Rate limit enforcement (60/10/5 tiers)           | ⚠️ 85.7% lines, lock-out untested     | Phase 1   |
| 3  | Login lockout after 10 failures → 15-min block   | ❌ Not tested                          | Phase 1   |
| 4  | WebSocket job status → real-time updates         | ❌ 18.8% coverage                      | Phase 2   |
| 5  | Backup create → verify → checksum validation     | ❌ 43.3% coverage, HAS BUGS            | Phase 0+3 |
| 6  | Manifest generate → lock → validate checksums    | ❌ 42.0% coverage, HAS BUGS            | Phase 0+3 |
| 7  | GPU node register → reserve → drain life-cycle   | ⚠️ API tested, service untested       | Phase 4   |
| 8  | Pipeline checkpoint → resume from failure        | ⚠️ API tested, service edge cases not | Phase 4   |
| 9  | Retention policy → tier migration report         | ⚠️ API tested, report calculation not | Phase 4   |
| 10 | Rollback point create → execute rollback         | ⚠️ 22.1% service coverage             | Phase 4   |

**After all phases, verify:**

- [ ] Each critical path has  $\geq 1$  happy-path test
- [ ] Each critical path has  $\geq 1$  error-path test
- [ ] Paths 2, 3, 5, 6 have bug fixes verified by tests

**Plan Change:**

- Add critical path verification as Phase 4 exit criteria
  - Map each path to specific test functions in v2 plan
-

## SECTION F: Phase 5 (Scripts) Justification

---

### F1: Is Phase 5 Padding?

**Position:** AGREE. Phase 5 is padding. Removing it.

**Honest assessment:**

`create_admin.py` , `seed_fallback_policies.py` , `seed_prompts.py` (134 lines total)  
are operational scripts run manually. They:

- Use `asyncio.run()` at module level → import fails in test context
- Require real database state → mocking defeats the purpose
- Are run once during setup → low operational risk

Testing them means mocking `asyncio.run()` and `AsyncSession.execute()` to verify that the mock was called with the right arguments. This proves the mock works, not that the script works.

**Coverage impact of removing Phase 5:**

v1 plan projected 83.0% with Phase 5. Without it:

- Phase 1-4: 81.1% (still above 80% threshold)
- Scripts' 134 lines stay at 0% but are excluded from critical coverage

**Plan Change:**

- Remove Phase 5 entirely
- If coverage falls below 80% after Phase 4, add targeted tests for actual coverage gaps — not script padding
- Mark scripts as “excluded from coverage targets” with justification

---

## SECTION G: Integration Tests

---

### G1: Integration Tests Should Be In Scope

**Position:** AGREE, with caveats.

**What unit tests with mocks prove:**

-  Logic correct in isolation
-  API contracts correct
-  Error handling works
-  External systems integrate correctly
-  End-to-end workflows work
-  Race conditions caught

**What integration tests would add:**

| Test Category            | What It Proves                      | Dependencies        |
|--------------------------|-------------------------------------|---------------------|
| Real Redis rate limiting | TTL actually expires                | Redis container     |
| Real Redis pub/sub       | Messages actually delivered         | Redis container     |
| Asset upload/download    | SeaweedFS round-trip                | SeaweedFS container |
| Cross-service workflow   | Project → Job → Checkpoint → Resume | All services + DB   |

#### Feasibility in this sandbox:

Docker:  Available (used for PG17)  
 Redis: Can run via docker run  
 SeaweedFS: Can run via docker run  
 Time: +18h estimated

#### Proposed Phase 6 scope:

| Test Group                | Tests     | Hours      | Priority                             |
|---------------------------|-----------|------------|--------------------------------------|
| Real Redis (TTL, pub/sub) | 8         | 6h         | HIGH — validates Phase 1             |
| Cross-service work-flows  | 6         | 6h         | MEDIUM — validates Phase 4           |
| Real SeaweedFS round-trip | 4         | 4h         | LOW — asset upload testable via mock |
| <b>Total</b>              | <b>18</b> | <b>16h</b> |                                      |

**Caveat:** Integration tests are SEPARATE from the 80% line coverage target.

They test system behavior, not code paths. They should be marked

`@pytest.mark.integration` and run separately.

#### Plan Change:

- Add Phase 6: Integration Tests (16-18h)
- Mark as `@pytest.mark.integration`
- Run after Phase 4 completes
- Do NOT count toward \$15.4 line coverage target
- Add to CI as separate job (run on merge to main, not every PR)

## G2: Reliability Claims

**Position:** AGREE. Must be explicit about what we can and cannot claim.

**After all phases complete:**

| Claim                       | Confidence | Evidence                                        |
|-----------------------------|------------|-------------------------------------------------|
| Unit logic correct          | HIGH       | 80%+ line coverage, 65%+ branch                 |
| API contracts work          | HIGH       | All endpoints tested                            |
| RBAC enforcement            | HIGH       | Existing + new tests                            |
| Rate limiting enforced      | HIGH       | Phase 1 + Phase 6 integration                   |
| WebSocket streaming works   | MEDIUM     | Depends on mock fidelity                        |
| External system integration | MEDIUM     | Phase 6 integration tests                       |
| Production deployment safe  | MEDIUM     | Migrations verified, but env differences remain |
| Performance acceptable      | LOW        | No load tests                                   |
| Concurrent access safe      | LOW        | Serial test execution only                      |

**Plan Change:** Add “Reliability Claims” section to v2 plan definition of done.

## SECTION H: Production Bug Discovery Handling

### H1: Bug Handling Process

**Position:** STRONGLY AGREE. And the urgency is higher than expected.

**Bugs Already Discovered During This Audit:**

| ID      | File                       | Severity | Type            | Description                                                                                                                                                                           |
|---------|----------------------------|----------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BUG-001 | backup.py:<br>292/299      | HIGH     | NameError       | <code>_exc</code> bound but <code>exc</code> referenced<br>→ crash in error handler                                                                                                   |
| BUG-002 | manifests.py:<br>85-92     | LOW      | Dead Code       | Unreachable <code>select("*")</code> call, result unused                                                                                                                              |
| BUG-003 | manifests.py:<br>99+       | HIGH     | Schema Mismatch | API uses <code>timeline_json</code> , <code>scene_count</code> , <code>created_at</code> — model has <code>timeline</code> , no <code>scene_count</code> , no <code>created_at</code> |
| BUG-004 | manifests.py:<br>176+      | HIGH     | Column Mismatch | API uses <code>sha256_hash</code> — model column is <code>content_hash</code>                                                                                                         |
| BUG-005 | backup.py:42+              | HIGH     | Column Mismatch | API uses <code>storage_path</code> — model column is <code>backup_path</code>                                                                                                         |
| BUG-006 | backup.py:43+              | HIGH     | Missing Column  | API uses <code>error_message</code> — model has no such column                                                                                                                        |
| BUG-007 | quality_service.py:<br>172 | MEDIUM   | Missing Attr    | Service sets <code>score.review_notes</code> — model has no <code>review_notes</code>                                                                                                 |
| BUG-008 | migrations/                | HIGH     | Migration Gap   | 5 model columns missing from Alembic migrations                                                                                                                                       |

**Impact:** BUG-003 through BUG-006 mean that `manifests.py` and `backup.py` **cannot function in production**. Their raw SQL references columns that don't exist in the ORM models or database schema. Any API call to these endpoints will produce a 500 error.

### Bug Handling Protocol:

#### DISCOVERY

- ☐ Document **in** BUGS\_FOUND.md (**file**, line, evidence, severity)
- ☐ **If HIGH**: immediately report **to** operator
- ☐ **If MEDIUM/LOW**: accumulate, report **at** phase boundary

#### TESTING

- ☐ **Write** test that EXPOSES the bug (`test_backup_error_handler_name_error`)
- ☐ **Mark**: `@pytest.mark.xfail(reason="BUG-001: NameError in error handler")`
- ☐ Bug fix **is** a SEPARATE **commit from** the test

#### GIT DISCIPLINE

- ☐ **test**: **add backup** API tests (exposes BUG-001, BUG-005, BUG-006)
- ☐ **fix**: BUG-001 ☐ rename `_exc` **to** `exc` **in** `backup.py` error handler
- ☐ **fix**: BUG-005 ☐ align `storage_path` ☐ `backup_path` **in** `backup.py`

#### ESCALATION

- ☐ **If** bug affects test **plan** (e.g., `can't test endpoint because it's broken`)☐  
report immediately, propose fix, wait **for** approval

### Plan Change:

- Create `BUGS_FOUND.md` as Phase 0 deliverable (DONE — findings above)
- All bugs documented before any fixes attempted
- Test commits separate from fix commits
- HIGH bugs require operator decision before fixing

## H2: Dead Code Investigation

**Position:** AGREE. Investigated.

### manifests.py lines 85-92:

```
_result = await db.execute( # noqa: F841
    select("*").select_from(
        __import__("sqlalchemy").text("composition_manifests")
    ).where(
        __import__("sqlalchemy").text("job_id = :job_id")
    ),
    {"job_id": job_id},
)
```

### Analysis:

- `_result` is assigned but never used ( `# noqa: F841` suppresses the warning)
- The real query starts on line 96 with `sa_text("SELECT ...")`
- The `__import__("sqlalchemy")` pattern is unusual — likely a quick hack
- `select("*")` is invalid SQLAlchemy 2.0 syntax (would raise at runtime)
- This code is dead because the function continues past it regardless

**Verdict:** Copy-paste artifact from development. The developer wrote this first attempt, then replaced it with raw SQL on line 96, but forgot to remove the first attempt.

**Recommendation:** Remove lines 85-92 in a separate fix commit. The `# noqa: F841` suppression is the smoking gun — developer knew it was unused.

**Plan Change:** Add to Phase 0c bug decisions.

## SECTION I: Existing Test Regression Protection

### I1: “153 Still Passing” Is Insufficient

**Position:** AGREE. Need per-file coverage stability check.

**Proposed Verification Script:**

```
# scripts/verify_regression.py
"""Compare coverage before/after fixture changes."""
import json, sys

baseline = json.load(open("coverage_baseline.json"))
current = json.load(open("coverage.json"))

regressions = []
for file in baseline["files"]:
    if file not in current["files"]:
        regressions.append(f"FILE REMOVED: {file}")
        continue

    base_pct = baseline["files"][file]["summary"]["percent_covered"]
    curr_pct = current["files"][file]["summary"]["percent_covered"]

    if curr_pct < base_pct - 0.5: # Allow 0.5% noise
        regressions.append(
            f"REGRESSION: {file}: {base_pct:.1f}% → {curr_pct:.1f}%"
        )

if regressions:
    print("✗ REGRESSIONS DETECTED:")
    for r in regressions:
        print(f"  {r}")
    sys.exit(1)
else:
    print("✓ No coverage regressions")
```

**When to run:**

1. After adding new fixtures to `conftest.py` (fixture leak detection)
2. After each phase completion
3. Before any git commit that modifies existing test files

**Plan Change:**

- Save `coverage_baseline.json` before Phase 1 starts
- Run regression check at every phase boundary
- If regression detected: investigate before continuing

---

## SECTION J: CI Integration

---

### J1: Expected Runtime & Optimization

**Position:** AGREE. Need projections.

**Current Runtime:**

- 153 tests: ~57 seconds (0.37s/test average)

**Projected Runtime After All Phases:**

| Category                | Tests      | Est. Time                  |
|-------------------------|------------|----------------------------|
| Existing                | 153        | 57s                        |
| Phase 1 (rate limiting) | 15         | 8s                         |
| Phase 2 (WebSocket)     | 12         | 15s (WS connect overhead)  |
| Phase 3 (API endpoints) | 33         | 15s                        |
| Phase 4 (service tests) | 89         | 40s                        |
| <b>Unit total</b>       | <b>302</b> | <b>~135s (2.3 min)</b>     |
| Phase 6 (integration)   | 18         | ~120s (container overhead) |
| <b>Full total</b>       | <b>320</b> | <b>~255s (4.3 min)</b>     |

Under 5 minutes — **no optimization needed** for CI.

**Parallel execution note:** Tests use TRUNCATE-and-commit pattern. Parallel execution ( `pytest-xdist` ) is not safe without per-worker database isolation. The tradeoff is acceptable: 4.3 min serial is fine for CI.

**Plan Change:** Document runtime projections. No parallelization needed. If runtime exceeds 5 min, consider session-scoped fixtures (not `xdist`).

---

### J2: CI Configuration Updates

**Position:** AGREE. CI needs updates.

**Required Changes:**

```
# .github/workflows/test.yml
test:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:17.2
      env:
        POSTGRES_USER: testuser
        POSTGRES_PASSWORD: testpass
        POSTGRES_DB: testdb
    redis:
      image: redis:7-alpine # For integration tests only

  steps:
    - name: Run unit tests
      run: |
        pytest tests/ \
          -m "not integration" \
          --cov=app \
          --cov-branch \
          --cov-report=term \
          --cov-report=xml \
          --cov-fail-under=80

    - name: Run integration tests
      if: github.ref == 'refs/heads/main'
      run: |
        pytest tests/ -m integration -v
```

**Plan Change:** Add CI configuration as a Phase 0 task (create/update workflow file).

---

## SECTION K: Sandbox-to-Production Transition

---

### K1: Environment Differences

**Position:** AGREE. Must be documented and mitigated.

**Known Differences:**

| Aspect                 | Sandbox       | Production (node-01) | Risk                             |
|------------------------|---------------|----------------------|----------------------------------|
| Python                 | 3.11.6        | 3.12.8               | LOW (no 3.12-only features used) |
| PostgreSQL             | 17.10         | 17.2 (Docker)        | NEGLIGIBLE (minor versions)      |
| TimescaleDB            | Not installed | Installed            | MEDIUM (hypertable tests skip)   |
| Redis                  | Mock (dict)   | Real (Docker)        | MEDIUM (TTL, pub/sub differ)     |
| SeaweedFS              | Mock          | Real (Docker)        | LOW (mock covers API surface)    |
| /ivgs/roll-back_points | chmod 777     | Proper UID/GID       | LOW                              |
| Test DB                | Local PG      | Docker PG            | LOW                              |

#### Mitigation for each:

1. **Python 3.11→3.12:** Run `python3.12 -m pytest tests/` before push (if pyenv available)
2. **TimescaleDB:** Mark hypertable tests as `@pytest.mark.skipif(not HAS_TIMESCALE)`
3. **Redis:** Phase 6 integration tests use real Redis → validates
4. **SeaweedFS:** Phase 6 integration tests use real SeaweedFS → validates
5. **Rollback dir:** Use `tempfile.mkdtemp()` in tests, not hardcoded path

#### Plan Change:

- Add environment compatibility section to v2 plan
- Tests use environment variables for all paths (no hardcoded `/ivgs/`)
- Phase 6 validates real service integration

## K2: Verification Checklist

**Position:** AGREE.

#### Sandbox Exit Checklist (before PR/push):

```
# 1. All unit tests pass
pytest tests/ -m "not integration" -v
# Expected: 302+ passed

# 2. Coverage meets threshold
pytest tests/ --cov=app --cov-branch --cov-fail-under=80
# Expected: ≥80% line, ≥65% branch

# 3. No regressions
python scripts/verify_regression.py
# Expected:  No coverage regressions

# 4. Integration tests pass (if Phase 6 complete)
pytest tests/ -m integration -v
# Expected: 18+ passed

# 5. Clean git state
git status # nothing to commit
git log --oneline -10 # clean history

# 6. No flaky tests (run 3x)
for i in 1 2 3; do pytest tests/ -q --tb=no || echo "FLAKY on run $i"; done
# Expected: 3/3 pass
```

#### node-01 Verification (after pull):

```
# 1. Pull and install
git pull origin main
pip install -r requirements.txt -r requirements-test.txt

# 2. Run migrations
alembic upgrade head

# 3. Run unit tests
pytest tests/ -m "not integration" -v

# 4. Run integration tests
pytest tests/ -m integration -v

# 5. Coverage check
pytest tests/ --cov=app --cov-branch --cov-report=term
```

#### Discrepancy Diagnostic:

If tests pass in sandbox but fail on node-01:

1. Compare Python versions: `python --version`
2. Compare installed packages: `pip freeze > deps.txt && diff`
3. Check database: `psql -c "SELECT version()"`
4. Check Redis: `redis-cli ping`
5. Run single failing test verbose: `pytest tests/test_foo.py::test_bar -vvs`

**Plan Change:** Add checklists to v2 plan as appendix.

---

## SUMMARY OF ALL CHANGES

### Structural Changes to Plan

| Change                                                  | Impact                 |
|---------------------------------------------------------|------------------------|
| <b>ADD Phase 0:</b> Schema migrations + bug decisions   | +12h, blocks Phase 3   |
| <b>ADD Phase 6:</b> Integration tests                   | +18h                   |
| <b>REMOVE Phase 5:</b> Script tests (padding)           | -6h                    |
| <b>REORDER:</b> Phase 0 → 1 → 2 → 3 → 4 → 6             | Dependencies respected |
| <b>RESIZE Phase 4:</b> Only services with coverage gaps | -10h                   |

### New Deliverables

| Deliverable                                 | Phase                             |
|---------------------------------------------|-----------------------------------|
| <code>SCHEMA_MIGRATION_GAP_REPORT.md</code> | Phase 0 (done in this document)   |
| <code>BUGS_FOUND.md</code>                  | Phase 0 (8 bugs documented above) |
| 5 new Alembic migrations                    | Phase 0                           |
| <code>scripts/verify_regression.py</code>   | Phase 0                           |
| Bug fix commits (BUG-001 through BUG-007)   | Phase 0c                          |
| WebSocket POC result                        | Phase 2 pre-gate                  |
| <code>coverage_baseline.json</code>         | Before Phase 1                    |

## Revised Timeline

| Phase                      | Work                                            | Hours         |
|----------------------------|-------------------------------------------------|---------------|
| Phase 0: Migrations + Bugs | Schema audit, 5 migrations, bug decisions       | 8-12h         |
| Phase 1: Rate Limiting     | 15 tests + Redis mock fix                       | 5-8h          |
| Phase 2: WebSocket         | POC + 12 tests + pub/sub mock                   | 6-10h         |
| Phase 3: API Endpoints     | 33 tests (blocked on Phase 0)                   | 7-10h         |
| Phase 4: Services          | 60-89 tests (scope depends on Phase 3 coverage) | 20-36h        |
| Phase 6: Integration       | 18 tests with real Redis/Sea-weedFS             | 12-18h        |
| <b>TOTAL</b>               | <b>~150-170 tests</b>                           | <b>58-94h</b> |

**Calendar time:** 2-4 weeks focused work (not 1 week as v1 implied).

## Risk Summary

| Risk                               | Probability | Impact        | Mitigation                        |
|------------------------------------|-------------|---------------|-----------------------------------|
| More schema bugs discovered        | HIGH        | +8h           | Phase 0 audit reduces surprise    |
| WebSocket POC fails                | MEDIUM      | +8h           | Fallback approaches documented    |
| Phase 4 scope larger than expected | MEDIUM      | +12h          | Re-assess after Phase 3 coverage  |
| Integration test containers fail   | LOW         | +4h           | Docker already working in sandbox |
| Coverage target unachievable       | LOW         | Plan revision | Phase 4 is largest contributor    |

## OPEN QUESTIONS FOR OPERATOR

1. **BUG-003 through BUG-006:** Fix column names in API (align to model) or add columns to model (align to API)? Both approaches are valid; need decision before Phase 3 tests can be written.

2. **Phase 6 scope:** Include SeaweedFS integration, or Redis-only?  
SeaweedFS adds 4h but tests a critical data path.
3. **Branch coverage target:** 65% overall acceptable, or push to 70%?  
Higher target adds ~10h to Phase 4.
4. **Bug fix authority:** Can agent fix HIGH bugs immediately, or must each fix be approved individually?
5. **Dead code (BUG-002):** Remove manifests.py lines 85-92, or leave?

## APPENDIX: Audit Raw Data

### A. Migration Files (14 migrations, 22 tables)

|                               |                                                                                                                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0001_initial_core.py          | → users, projects, transcripts, storyboard_scenes, assets, prompts, prompt_tags, prompt_tag_associations, render_jobs, language_variants, audit_log, roll-back_points |
| 0002_pipeline_checkpoints.py  | → pipeline_checkpoints                                                                                                                                                |
| 0003_gpu_registry.py          | → gpu_nodes, gpu_reservations                                                                                                                                         |
| 0004_retry_tracking.py        | → task_retries                                                                                                                                                        |
| 0005_worker_heartbeats.py     | → worker_heartbeats                                                                                                                                                   |
| 0006_dead_letter_queue.py     | → dead_letter_messages                                                                                                                                                |
| 0007_composition_manifests.py | → composition_manifests                                                                                                                                               |
| 0008_quality_scores.py        | → asset_quality_scores                                                                                                                                                |
| 0009_render_segments.py       | → render_segments                                                                                                                                                     |
| 0010_gpu_metrics.py           | → gpu_metrics_history                                                                                                                                                 |
| 0011_retention_policies.py    | → retention_policies                                                                                                                                                  |
| 0012_storage_quotas.py        | → storage_quotas                                                                                                                                                      |
| 0013_backup_records.py        | → backup_records                                                                                                                                                      |
| 0014_fallback_policies.py     | → fallback_policies                                                                                                                                                   |

### B. Column Gap Evidence

#### users.is\_active:

- Model: `is_active: Mapped[bool] = mapped_column(Boolean, ...)`
- Migration 0001: Columns are `id`, `username`, `password_hash`, `role`, `created_at`, `last_login_at`
- `is_active` NOT present in any migration

#### projects.created\_by:

- Model: `created_by: Mapped[Optional[uuid.UUID]] = mapped_column(UUID, ForeignKey("users.id"))`
- Migration 0001: Columns are `id`, `name`, `description`, `max_runtime_seconds`, `state`, `hero_image_asset_id`, `talking_head_asset_id`, `target_audience`, `created_at`, `updated_at`
- `created_by` NOT present in any migration

#### asset\_quality\_scores.job\_id:

- Model: `job_id: Mapped[Optional[uuid.UUID]] = mapped_column(UUID, ForeignKey("render_jobs.id"))`
- Migration 0008: Columns are `id`, `asset_id`, `quality_score`, `safety_score`, `scoring_details`, `decision`, `reviewed_by`, `reviewed_at`, `created_at`
- `job_id` NOT present in any migration

**retention\_policies.description:**

- Model: `description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)`
- Migration 0011: Columns are `id, name, hot_days, warm_days, cold_days, archive_days, delete_after_days, applies_to, is_default, created_at, updated_at`
- `description` NOT present in any migration

**prompt\_tags.description:**

- Model: `description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)`
- Migration 0001 prompt\_tags: Columns are `id, name, created_at`
- `description` NOT present in any migration

**C. Service Classification Raw Data**

|                                 |                                                                |                              |
|---------------------------------|----------------------------------------------------------------|------------------------------|
| <code>checkpoint_service</code> | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>dlq_service</code>        | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>gpu_service</code>        | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>job_service</code>        | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>language_service</code>   | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>prompt_service</code>     | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>quality_service</code>    | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>retention_service</code>  | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>storyboard_service</code> | <code>→ __init__(self, db: AsyncSession)</code>                | <code>→ PURE_DB</code>       |
| <code>asset_service</code>      | <code>→ __init__(self, db: AsyncSession) + seaweedfs</code>    | <code>→ DB+EXTERNAL</code>   |
| <code>transcript_service</code> | <code>→ __init__(self, db: AsyncSession) + seaweedfs</code>    | <code>→ DB+EXTERNAL</code>   |
| <code>project_service</code>    | <code>→ __init__(self, db: AsyncSession) + current_user</code> | <code>→ DB+EXTERNAL</code>   |
| <code>rollback_service</code>   | <code>→ __init__(self) + subprocess + filesystem</code>        | <code>→ EXTERNAL_ONLY</code> |
| <code>auth_service</code>       | <code>→ uses redis_client (module-level)</code>                | <code>→ EXTERNAL_ONLY</code> |
| <code>user_service</code>       | <code>→ static/class methods, no __init__</code>               | <code>→ PURE_LOGIC</code>    |

---

End of revision document. Addresses all 22 review items (A1-K2) with evidence from the actual codebase. Ready for operator review.